

Université Cadi Ayyad (UCA) — École Nationale des Sciences
Appliquées (ENSA)

Marrakech, Maroc | Génie CyberDefense et systèmes de télécom embarqués

ARTICLE DE RECHERCHE — PUBLICATION LOGICIELLE

VaultDex

SecureStorageInspector

Architecture et Conception d'un Scanner de Sécurité Automatisé pour Applications Android

Abdelhakim AMHIRAQ Anas AOURIK Lahsen AIT OIHMANE Sultan ZEINEDDINE

École Nationale des Sciences Appliquées, Université Cadi Ayyad, Marrakech, Maroc

Encadrant : Professeur Lachgar

Emails : l.aitoihmane7493@uca.ac.ma | m.lachgar@uca.ac.ma

Python 3.13

FastAPI

React 19

Celery + Redis

PostgreSQL 16

Abstract

La prolifération des applications Android a engendré des risques significatifs liés au stockage insécurisé de données sensibles. Les développeurs exposent fréquemment, de manière involontaire, des mots de passe en clair, des tokens d'API, des informations personnellement identifiables (PII) et des clés cryptographiques directement dans le stockage local de leurs applications. L'audit manuel de ces vulnérabilités est un processus long, technique et sujet à erreurs humaines qui peut requérir plusieurs heures par application et exige une maîtrise approfondie des mécanismes internes d'Android.

VaultDex (SecureStorageInspector) est une plateforme web complète qui automatise intégralement ce processus d'audit de sécurité mobile. La solution déploie un fichier APK Android sur un émulateur Genymotion via ADB (Android Debug Bridge), permet à l'utilisateur d'interagir naturellement avec l'application cible afin de générer des données de stockage réalistes et représentatives d'un usage réel, puis extrait et analyse l'intégralité du stockage local au travers d'un **moteur de 28 règles de sécurité** catégorisées couvrant les SharedPreferences, les bases de données SQLite, les fichiers génériques et le cache applicatif. Elle produit un rapport de sécurité détaillé assorti d'un score de risque normalisé (0–100) et d'un export PDF généré côté client.

L'architecture repose sur une pile technologique moderne : **FastAPI** (backend asynchrone ASGI), **React 19** (frontend SPA avec SSE en temps réel), **Celery + Redis** (traitement asynchrone distribué) et **PostgreSQL 16** (persistance relationnelle multi-tenant), tous orchestrés via **Docker Compose**.

La validation sur une application Android conçue à cet effet (com.vaultdex.app, version 1.0, SDK 26/34, 16,28 Mo) a permis de détecter **87 problèmes de sécurité répartis dans 15 fichiers**, produisant un score de **100/100 (Risque Critique)**, dont 30 vulnérabilités critiques d'exposition de tokens d'authentification en clair, 49 hautes, 6 moyennes et 2 basses.

Mots-clés : Sécurité Android, Analyse APK, ADB, SharedPreferences, SQLite, FastAPI, Celery, React, SSE, OWASP MASTG, JWT, Multi-tenant, Audit Automatisé, Genymotion.

Table 1: Métadonnées du code et du logiciel

Nr.	Description	Valeur
C1	Version actuelle	1.0.0
C2	Dépôt de code	https://github.com/LahsenAitOiahmane/vaultdex.web
C3	Licence	Licence MIT
C4	Gestionnaire de versions	Git
C5	Langages et services	Python 3.13+, TypeScript 5, SQL, Bash
C6	Dépendances principales	FastAPI, SQLAlchemy 2.0, Pydantic v2, Celery 5, Redis 7, PostgreSQL 16, React 19, Vite 8, TailwindCSS v4, Axios, Recharts, jsPDF, Genymotion Desktop, SDK Android (ADB, aapt2), defusedxml, slowapi, python-jose, passlib[bcrypt]
C7	Contact	m.lachgar@uca.ac.ma l.aitoihmane7493@uca.ac.ma

1 Introduction

1.1 Contexte et Motivation

La sécurité des applications mobiles est devenue un enjeu stratégique majeur dans le paysage numérique contemporain. Selon les données publiées par Statista en 2025, plus de 3,5 millions d'applications sont disponibles sur le Google Play Store, et plusieurs milliards d'appareils Android sont en circulation dans le monde. Cette omniprésence du mobile dans les usages quotidiens — banque en ligne, santé numérique, commerce électronique, réseaux sociaux — a fait de la sécurité des données mobiles un enjeu critique pour les entreprises, les développeurs et les utilisateurs finaux.

Android offre plusieurs mécanismes de stockage local que les développeurs utilisent fréquemment de manière inappropriée. Les **SharedPreferences** — fichiers XML clé-valeur situés dans `shared_prefs/` — sont couramment utilisés pour stocker des tokens d'authentification, des mots de passe et des clés API en clair. Les **bases de données SQLite** (`databases/`) contiennent des données structurées persistantes (profils utilisateurs, historiques, transactions) souvent sans chiffrement SQLCipher. Le **répertoire de fichiers** et le **cache applicatif** accumulent des données sensibles résiduelles issues de l'exploitation normale de l'application.

Selon le rapport OWASP Mobile Security Testing Guide (MASTG) 2024 [1], la vulnérabilité *M9 – Stockage de données non sécurisé* figure systématiquement parmi les cinq principales menaces du Mobile Top 10. Une application vulnérable peut exposer les données de ses utilisateurs lors d'analyses forensiques, de compromissions d'appareils rootés, ou via des sauvegardes ADB non protégées (`allowBackup=true`).

1.2 Problématique

Question de recherche centrale : Comment automatiser de façon fiable et reproductible l'inspection du stockage local d'une application Android, en combinant l'analyse statique du manifeste APK et l'analyse dynamique post-exécution réelle, au sein d'une plateforme web sécurisée, accessible et multi-tenant ?

Les outils existants répondent chacun à une partie seulement de ce besoin. **MobSF** [3] offre une analyse statique et dynamique, mais son analyse dynamique repose sur des scripts automatisés incapables de simuler un usage réel authentique. **Frida** et **Drozer** sont des frameworks d'instrumentation puissants mais exclusivement en ligne de commande, sans interface web ni rapport standardisé. **APKTool** et **jadx** se limitent à la décompilation statique sans inspection du stockage runtime.

1.3 Contributions du Projet

VaultDex apporte les contributions techniques suivantes :

1. Un **pipeline hybride en deux phases** combinant analyse statique (`aapt2`) et analyse dynamique post-interaction réelle, avec une fenêtre d'interaction manuelle garantissant la collecte de données de stockage authentiques.

2. Un **moteur de détection à 28 règles extensible**, organisé en quatre modules indépendants couvrant les catégories credentials, PII, cryptographie et configuration.
3. Une **plateforme web multi-tenant** avec authentification JWT et isolation stricte des données par `owner_id`, permettant l'usage simultané par plusieurs analystes.
4. Une **interface SSE temps réel** et un export PDF côté client (jsPDF) assurant la confidentialité des données de findings.

1.4 Structure du Document

Cet article est organisé comme suit : la Section 2 présente les objectifs et fonctionnalités ; la Section 3 détaille l'architecture et les trois diagrammes de modélisation ; la Section 4 décrit les technologies utilisées ; la Section 5 présente un exemple d'utilisation illustratif ; la Section 6 aborde la sécurité de la plateforme ; la Section 7 traite du déploiement ; la Section 8 couvre les tests ; et la Section 9 conclut avec les perspectives.

2 Objectifs et Fonctionnalités Principales

2.1 Objectifs Généraux

VaultDex poursuit quatre objectifs principaux :

1. **Automatiser** l'audit de sécurité du stockage local des APK Android sans nécessiter d'expertise technique préalable de l'analyste de sécurité.
2. **Centraliser** les résultats de scans dans une plateforme web multi-utilisateurs sécurisée avec historique persistant.
3. **Standardiser** l'évaluation des risques via un score normalisé (0–100) et une classification de sévérité en quatre niveaux (CRITIQUE / ÉLEVÉ / MOYEN / FAIBLE).
4. **Faciliter** la communication des résultats via un rapport PDF exportable directement depuis le navigateur, sans charge serveur.

2.2 Fonctionnalités Principales

2.2.1 Analyse Statique du Manifeste APK

L'analyse statique opère directement sur le fichier APK avant toute interaction avec l'émulateur. Elle invoque `aapt2 dump badging` pour décoder le manifeste `AndroidManifest.xml` binaire et en extraire : la liste complète des permissions déclarées (`uses-permission`) ; les composants exportés sans protection par permission (`android:exported="true"`) ; et les drapeaux de sécurité critiques : `android:debuggable`, `android:allowBackup`, `android:usesCleartextTraffic`.

2.2.2 Analyse Dynamique Post-Exécution

L'analyse dynamique constitue le cœur différenciant de VaultDex. Après l'installation de l'APK sur l'émulateur Genymotion via `adb install`, l'analyste interagit librement avec l'application — connexion, navigation, saisie de données personnelles, réalisation de transactions — générant ainsi des données de stockage représentatives d'un usage réel. Phase B extrait ensuite l'intégralité du répertoire `/data/data/package/` via `adb pull` et soumet chaque fichier aux cinq analyseurs spécialisés.

2.2.3 Moteur de Détection — 28 Règles de Sécurité

Table 2: Catégories et exemples de règles du moteur de détection VaultDex

Catégorie	Module Python	Exemples de motifs détectés
Credentials	<code>credential_rules.py</code>	Mots de passe en clair, tokens d'API, secrets OAuth, clés secrètes hardcodées, tokens Bearer JWT, credentials SMTP, matériel de clé SSH
PII	<code>pii_rules.py</code>	Adresses e-mail (RFC-5322), numéros de téléphone, numéros de carte bancaire (validation Luhn), IBAN, numéros d'identité nationaux
Cryptographie	<code>crypto_rules.py</code>	Hachages MD5 et SHA1, clés RSA/AES en clair, vecteurs d'initialisation fixes, algorithmes faibles DES/3DES, graine PRNG prévisible
Configuration	<code>config_rules.py</code>	Endpoints HTTP non HTTPS, <code>debug=true</code> , <code>allowBackup=true</code> , URLs d'environnement de développement en production

2.2.4 Reporting et Interface Utilisateur

L'interface utilisateur, développée en React 19, offre les fonctionnalités suivantes : un **tableau de bord** affichant les statistiques globales de l'utilisateur (nombre de scans, distribution des sévérités via Recharts) ; une **page de progression SSE** mettant à jour la timeline du pipeline en temps réel sans polling ; une **page de rapport** présentant le score de risque, la répartition sévérité, le détail par zone de stockage et la liste complète des findings ; et un **export PDF** généré intégralement côté client via jsPDF.

2.3 Mesures de Sécurité de la Plateforme

Table 3: Mesures de sécurité de la plateforme VaultDex

Mesure	Implémentation
Authentification	JWT signé HMAC-SHA256 + hachage bcrypt adaptatif, expiration 7 jours (<code>python-jose</code>)
Isolation des données	Filtrage systématique par <code>owner_id</code> sur toutes les requêtes base de données (<code>crud.py</code>)
Upload sécurisé	Validation triple : magic bytes APK (0x504B0304), extension <code>.apk</code> , taille max 100 Mo
Nommage des fichiers	UUID v4 aléatoires empêchant le path traversal et les collisions entre utilisateurs
Rate limiting	5 uploads/minute par adresse IP (bibliothèque <code>slowapi</code>)
Protection XML	<code>defusedxml</code> contre les attaques XXE (XML External Entity)
Headers HTTP	<code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> , Content Security Policy, HSTS (<code>middleware.py</code>)
CORS	Origines autorisées configurables via <code>ALLOWED_ORIGINS</code>

2.4 Flux de Scan en Deux Phases

Table 4: Description des deux phases du pipeline de scan VaultDex

Phase	Étapes	Description
Phase A	INIT → STATIC → RESET → BOOT → INSTALL → LAUNCH → WAITING	Analyse statique du manifeste via <code>aapt2</code> , reset snapshot VM, boot Android, installation ADB, lancement application. Attente de l'interaction utilisateur.
Phase B	PULL → CLEANUP → ZIP → ANALYSE → REPORT → COMPLETED	Extraction ADB du répertoire applicatif, application des 28 règles, calcul du score de risque, persistance du rapport JSON en PostgreSQL.

3 Architecture du Système et Diagrammes

3.1 Description de l'Architecture Globale

VaultDex adopte une architecture **microservices découplée en quatre couches fonctionnelles**. Ce choix architectural garantit la séparation des responsabilités, la scalabilité indépendante de chaque composant, et la maintenabilité à long terme du système.

La **couche présentation** est une application Single Page (SPA) React 19 compilée avec Vite 8, communiquant avec l'API via un client Axios configuré avec des intercepteurs JWT automatiques. La progression des scans est suivie en temps réel via le protocole SSE (Server-Sent Events), sans recours aux WebSockets. La **couche API** est une API RESTful asynchrone (ASGI) construite avec FastAPI, gérant l'authentification, la validation des uploads, les opérations CRUD et la délégation des tâches longues au worker via Redis. La **couche worker** exécute le pipeline d'analyse de manière asynchrone via Celery, orchestrant ADB, le moteur d'analyse statique et les cinq analyseurs spécialisés. La **couche infrastructure** regroupe PostgreSQL 16, Redis 7 et Genymotion Desktop, tous sécurisés en accès local uniquement.

3.2 Diagramme d'Architecture Technique

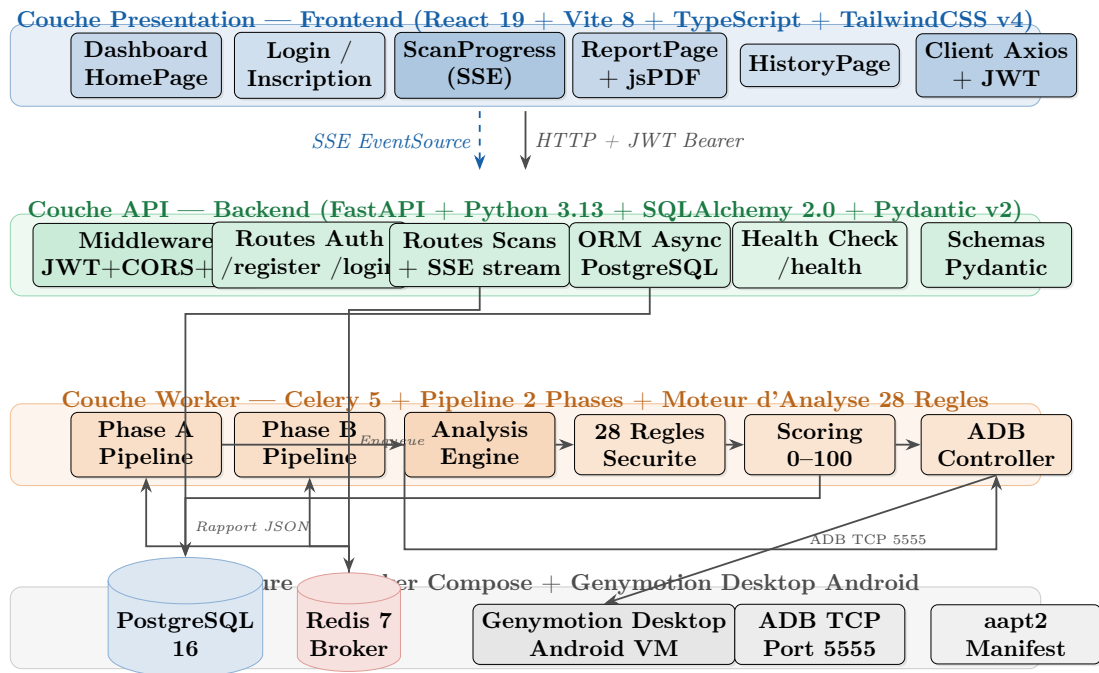


Figure 1: Diagramme d'architecture technique de VaultDex (vue en couches). Le frontend React communique avec l'API FastAPI via HTTP REST + JWT, et maintient un flux SSE pour la progression. L'API délègue les scans à Celery via Redis. Le worker orchestre ADB, analyse le stockage, calcule le score et persiste en PostgreSQL.

3.3 Diagramme de Cas d'Utilisation

Trois acteurs principaux interagissent avec VaultDex. Le **Visiteur** (non authentifié) peut uniquement s'inscrire ou se connecter. L'**Utilisateur Authentifié** accède au cycle complet de scan : soumission d'APK, suivi de progression, interaction manuelle avec l'application

Android, finalisation et consultation du rapport. Le **Système Worker** (acteur automatisé Celery) orchestre le pipeline interne, applique les règles et notifie le frontend via SSE.

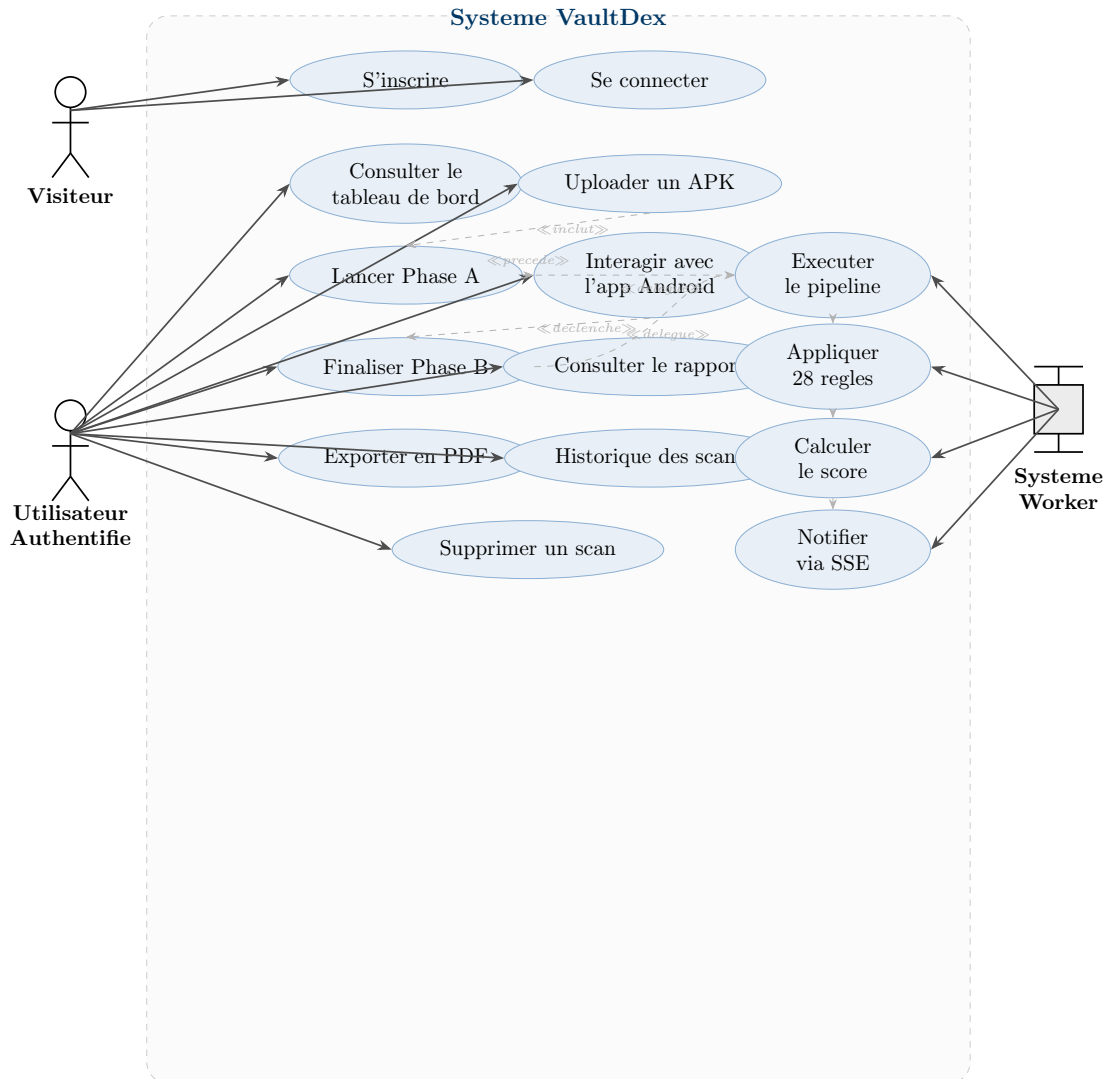


Figure 2: Diagramme de cas d'utilisation de VaultDex. La chaîne *Upload* → *Phase A* → *Interaction* → *Phase B* illustre le flux séquentiel obligatoire d'un scan.

3.4 Diagramme de Séquence — Cycle Complet d'un Scan APK

Le diagramme de séquence ci-dessous illustre l'ensemble des interactions entre les composants lors d'un cycle de scan complet, depuis l'authentification initiale jusqu'à l'export PDF du rapport. Quatre blocs distincts structurent le flux : l'authentification JWT, la Phase A d'installation, l'interaction manuelle de l'analyste, et la Phase B d'extraction et d'analyse.

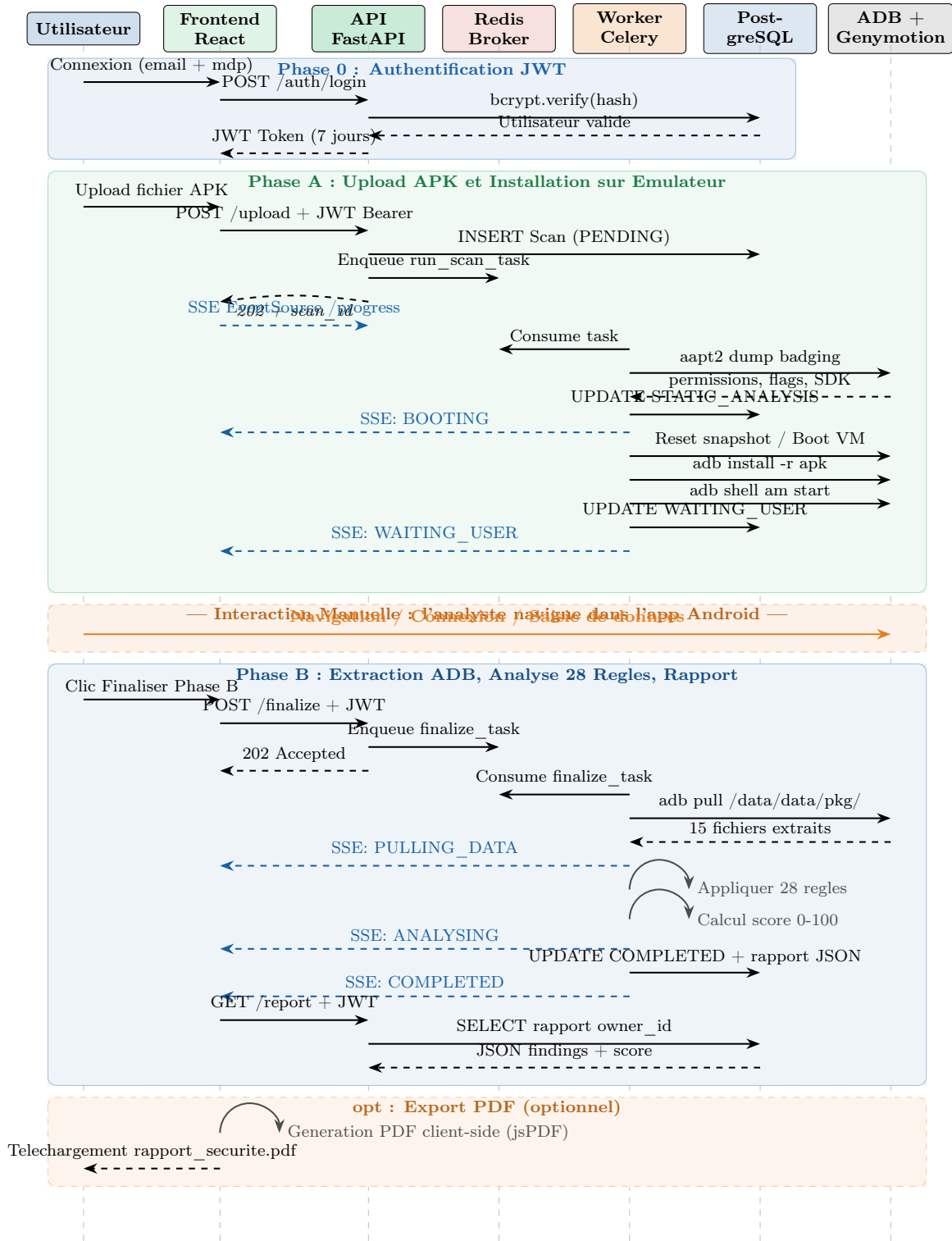


Figure 3: Diagramme de séquence — Cycle complet d'un scan VaultDex. Phase 0 : authentification JWT. Phase A : upload, analyse statique, boot VM, installation ADB. Interaction manuelle de l'analyste. Phase B : extraction ADB, 28 règles, score, rapport. Chaque transition émet un événement SSE vers le frontend.

3.5 Schéma de la Base de Données (UML / ERD)

Le diagramme entité-association ci-dessous détaille la structure relationnelle de la base de données PostgreSQL gérée par l'ORM SQLAlchemy. La plateforme repose sur deux tables

principales : **users** pour la gestion de l'authentification et l'isolation multitenant, et **scans** pour le suivi du cycle de vie complet d'un audit de sécurité.

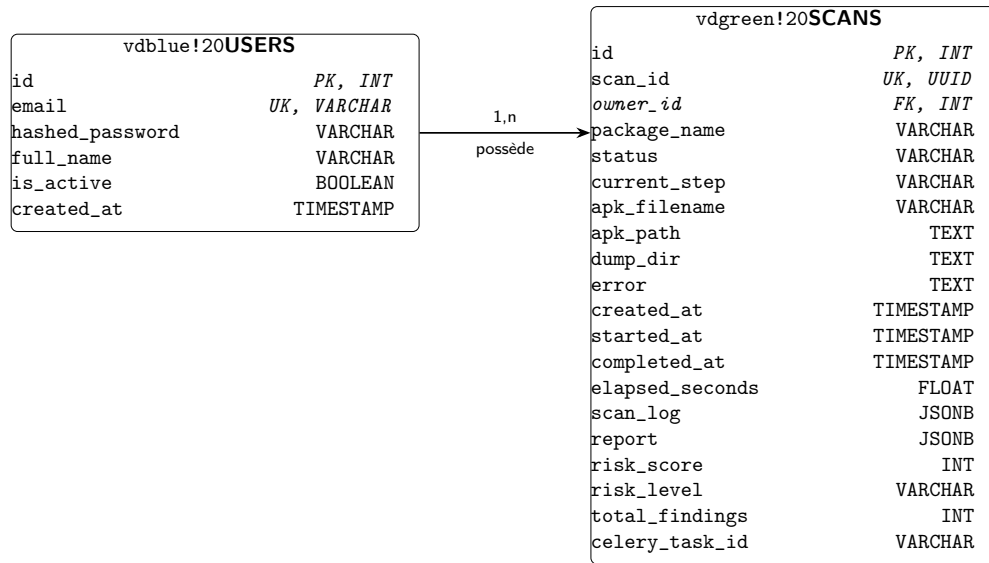


Figure 4: Diagramme Entité-Association de la base de données VaultDex. Un utilisateur (tenant) peut posséder plusieurs scans, garantissant l'isolation stricte des données d'audit via la clé étrangère `owner_id`. Le champ `report` stocke le rapport complet sous forme de document JSONB indexable.

4 Technologies Utilisées

4.1 Backend — FastAPI et Écosystème Python

Table 5: Technologies backend de VaultDex

Technologie	Version	Rôle dans VaultDex
Python	3.13+	Langage principal du backend et du worker
FastAPI	0.100+	Framework API asynchrone ASGI, documentation OpenAPI auto-générée
SQLAlchemy	2.0	ORM asynchrone avec driver asyncpg (<code>models.py</code>)
Pydantic	v2	Validation stricte des schémas requête/réponse (<code>schemas.py</code>)
Celery	5.x	Gestionnaire de tâches distribuées asynchrones (<code>celery_app.py</code>)
python-jose	3.x	Génération et validation des tokens JWT (<code>auth.py</code>)
passlib bcrypt	+ —	Hachage sécurisé adaptatif des mots de passe
slowapi	—	Rate limiting par IP : 5 uploads/minute (<code>middleware.py</code>)
defusedxml	—	Parsing XML sécurisé contre XXE (<code>sharedprefs_analyser.py</code>)
aiofiles	—	Écriture asynchrone non-bloquante lors des uploads APK

FastAPI a été choisi pour ses performances ASGI natives, sa génération automatique de documentation OpenAPI (accessible sur `/docs` et `/redoc`), et son intégration native avec Pydantic v2. L'ORM SQLAlchemy 2.0 en mode asynchrone avec le driver `asyncpg` garantit que toutes les opérations en base de données sont non-bloquantes, essentiel pour une API gérant des uploads de 100 Mo de manière concurrente.

4.2 Worker et Infrastructure Asynchrone

Table 6: Technologies d’infrastructure et de worker

Technologie	Version	Rôle dans VaultDex
Celery	5.x	Orchestration du pipeline d’analyse en 2 phases (<code>scan_pipeline.py</code>)
Redis	7 (Alpine)	Broker de messages Celery + backend de résultats
PostgreSQL	16 (Alpine)	Base relationnelle ACID : tables <code>users</code> et <code>scans</code>
Docker Compose	24+	Conteneurisation PostgreSQL et Redis (bind 127.0.0.1)
Genymotion Desktop	3.x	Émulateur Android x86 avec gestion des snapshots
ADB	SDK 34+	Contrôle de l’émulateur via TCP (<code>install</code> , <code>pull</code> , <code>am start</code>)
aapt2	SDK 34+	Décodage du manifeste APK binaire (<code>static_analyser.py</code>)

Le pattern Celery + Redis découple totalement l’API du traitement potentiellement long (plusieurs minutes par scan), évitant les timeouts HTTP et les connexions idle. La concurrence du worker est limitée à `-concurrency=1` pour éviter les conflits ADB avec l’émulateur unique. PostgreSQL a été préféré à une solution NoSQL pour ses garanties ACID et la richesse des requêtes analytiques (statistiques par utilisateur, historique filtré par date).

4.3 Frontend — React 19 et TypeScript

Table 7: Technologies frontend de VaultDex

Technologie	Version	Rôle dans VaultDex
React	19	Bibliothèque UI, composants fonctionnels avec hooks
TypeScript	5.x	Typage statique, interfaces des types API (<code>api/client.ts</code>)
Vite	8.x	Bundler ultra-rapide (HMR, ESM natif)
TailwindCSS	v4	Framework CSS utilitaire, design system cohérent
Axios	—	Client HTTP avec intercepteurs JWT automatiques
Recharts	—	Graphiques interactifs de distribution des findings
Framer Motion	—	Animations fluides de transitions entre les pages
jsPDF	—	Génération PDF côté client sans charge serveur (<code>ReportPage.tsx</code>)
Lucide React	—	Bibliothèque d’icônes SVG pour l’interface utilisateur

Le **Server-Sent Events (SSE)** a été préféré aux WebSockets pour le suivi de progression : flux unidirectionnel serveur → client, plus simple à implémenter, compatible avec les proxies HTTP standard et natif dans les navigateurs via **EventSource**. La génération PDF avec jsPDF côté client élimine la charge serveur et préserve la confidentialité des données de sécurité.

4.4 Récapitulatif de la Pile Technologique

Pile technologique complète de VaultDex

Couche	Technologies
Frontend	React 19 · TypeScript 5 · Vite 8 · TailwindCSS v4 · jsPDF
API Backend	FastAPI · Python 3.13 · Pydantic v2 · SQLAlchemy 2.0
Worker	Celery 5 · scan_pipeline · AnalysisEngine · 28 règles
Broker / Cache	Redis 7 (Alpine)
Base de données	PostgreSQL 16 · asyncpg · UUID PKs
Analyse APK	aapt2 · ADB · Genymotion Desktop · defusedxml
Infrastructure	Docker Compose · Licence MIT

5 Exemple d'Utilisation Illustratif : com.vaultdex.app

5.1 Description de l'Application de Test

Pour valider la plateforme, une application Android conçue spécifiquement à cet effet (`com.vaultdex.app`) a été développée pour contenir un ensemble représentatif de patterns de stockage non sécurisé couvrant tous les mécanismes supportés par le moteur d'analyse VaultDex. Ses caractéristiques sont : nom de package `com.vaultdex.app`, version 1.0 (code 1), SDK minimum 26 (Android 8.0), SDK cible 34 (Android 14), ABIs supportées `arm64-v8a`, `armeabi-v7a`, `x86`, `x86_64`, activité principale `MainActivity`, taille 16,28 Mo. L'application implémente intentionnellement le stockage de tokens d'authentification et de clés API en clair dans les `SharedPreferences`, des identifiants non chiffrés en `SQLite`, des valeurs de configuration sensibles dans des fichiers `JSON`, et plusieurs indicateurs de manifeste non sécurisés.

5.2 Phase A : Analyse Statique et Installation

La figure 5 montre l'interface VaultDex durant l'exécution de la Phase A. Le panneau gauche affiche la timeline de progression avec les indicateurs d'état de chaque étape ; le panneau terminal droit diffuse en temps réel le journal du worker Celery transmis via SSE.

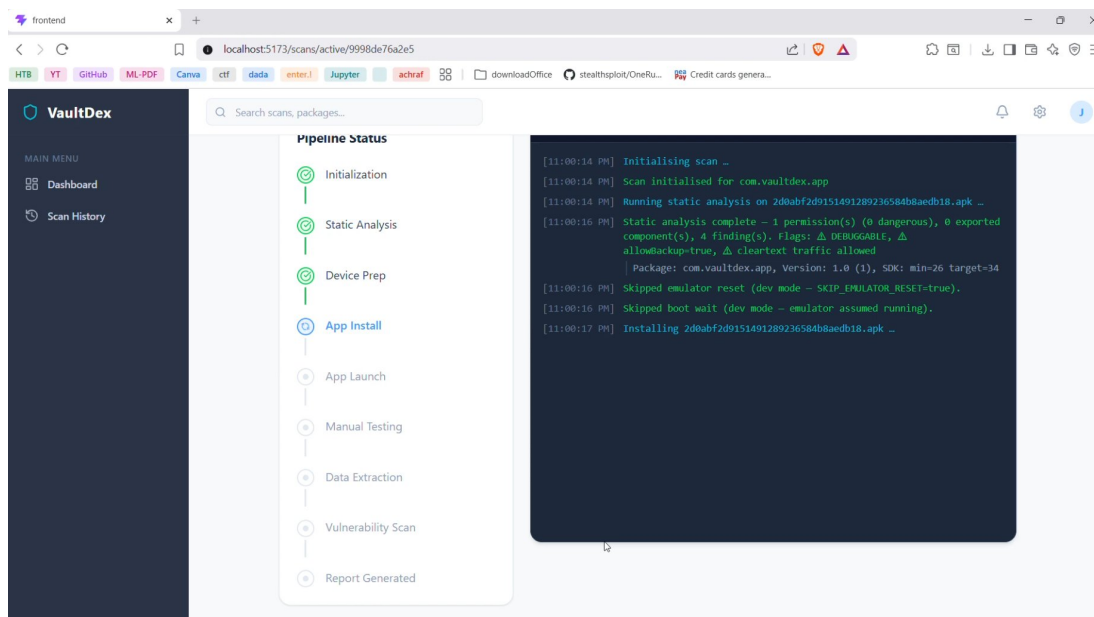


Figure 5: Interface VaultDex — Vue Pipeline Status durant la Phase A (étape **App Install** en cours d'exécution). Les étapes Initialization, Static Analysis et Device Prep sont terminées (coché vert). Le journal terminal affiche : analyse statique complète — 1 permission (0 dangereuse), 0 composant exporté, 4 findings manifeste incluant `DEBUGGABLE`, `allowBackup=true` et `cleartext traffic` autorisé, SDK min=26 target=34. Le worker procède à l'installation de l'APK.

L'étape d'analyse statique (`STATIC_ANALYSIS`) s'est achevée en moins de 2 secondes et a produit les findings manifeste suivants pour `com.vaultdex.app` : (1) drapeau `DEBUGGABLE=true` — sévérité ÉLEVÉE (MASVS-CODE-2) ; (2) drapeau `allowBackup=true` — sévérité ÉLEVÉE (MASVS-STORAGE-1) ; (3) trafic `cleartext` autorisé — sévérité MOYENNE (MASVS-NETWORK-1) ; (4) SDK minimum inférieur au niveau recommandé — sévérité FAIBLE.

5.3 Phase B : Analyse Dynamique et Rapport de Sécurité

Après que l'analyste a navigué dans l'application et déclenché plusieurs flows d'authentification, la Phase B a été initiée. Le worker a extrait 15 fichiers depuis `/data/data/com.vaultdex.app/` via `adb pull`, appliqué les 28 règles et calculé le score de risque. La figure 6 présente le tableau de bord du rapport généré.

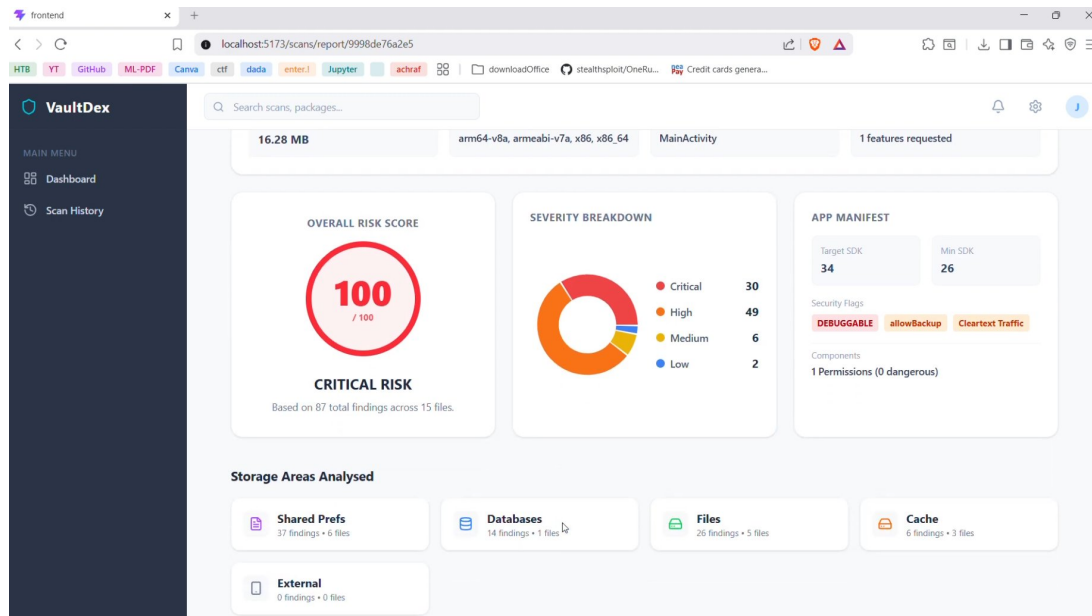


Figure 6: Tableau de bord du rapport VaultDex pour `com.vaultdex.app` (Scan ID : 9998de76a2e5). **Score de Risque Global : 100/100 (RISQUE CRITIQUE)**, 87 findings au total dans 15 fichiers. Distribution de sévérité : Critique 30, Élevé 49, Moyen 6, Faible 2. Zones de stockage analysées : SharedPrefs (37 findings, 6 fichiers), Databases (14 findings, 1 fichier), Files (26 findings, 5 fichiers), Cache (6 findings, 3 fichiers). Section Manifeste : SDK target 34 / min 26, drapeaux de sécurité **DEBUGGABLE**, **allowBackup**, **ClearText Traffic**.

Table 8: Distribution des findings et mapping OWASP MASTG pour `com.vaultdex.app` (scan 9998de76a2e5, 23/05/2026 23:00 UTC)

Zone de stockage	Fichiers	Findings	Contrôle MASVS	Catégorie principale
SharedPreferences XML	6	37	MASVS-STORAGE-1	Credentials, PII
Bases SQLite	1	14	MASVS-STORAGE-2	Credentials, PII
Fichiers génériques	5	26	MASVS-STORAGE-1	Credentials, Config
Cache applicatif	3	6	MASVS-STORAGE-2	PII, Crypto
Stockage externe	0	0	—	—
Drapeaux manifeste	—	4	MASVS-CODE-2	Configuration
Total	15	87		
Critique (pénalité 25)		30	Tokens auth. en clair, clés API	
Élevé (pénalité 10)		49	Clés API, mots de passe, PII	
Moyen (pénalité 5)		6	Algorithmes faibles, config cleartext	
Faible (pénalité 1)		2	Observations de configuration mineures	
Score calculé :			$100 - \min(100, 30 \times 25 + 49 \times 10 + 6 \times 5 + 2 \times 1) = 0 \rightarrow 100/100$	

Les findings les plus critiques proviennent du fichier `shared_prefs/api_keys.xml`, qui contenait quatre tokens d'authentification et clés API distincts en clair sous les noms de clé `auth_token`, `api_key`, `stripe_key` et `firebase_server_key`. Chacun a reçu la sévérité CRITIQUE, les tokens d'authentification étant équivalents à des identifiants de session capables d'accorder un accès complet au compte si exfiltrés.

5.4 Export du Rapport PDF

La figure 7 présente le rapport PDF exporté directement depuis le navigateur via jsPDF. L'en-tête inclut l'identifiant de scan, l'horodatage d'analyse, le score global et la répartition par sévérité. La table des findings présente pour chaque entrée la sévérité colorisée, le nom de la règle déclenchée, la localisation précise dans le stockage, et une description détaillée avec recommandation de remédiation.

Security Report

VaultDex • Scan ID: 9998de76a2e5

Analysed At: 5/23/2026, 11:00:56 PM

Overall Risk Score: 100 / 100

Risk Level: CRITICAL RISK

Critical: 30 | High: 49 | Medium: 6 | Low: 2

Severity	Rule	Location	Description
CRITICAL	Plaintext Auth Token	shared_prefs - api_keys.xml	An authentication or bearer token was found stored in plaintext. Tokens are equivalent to session credentials and must be protected with EncryptedSharedPreferences or Android Keystore.
CRITICAL	Plaintext API Key	shared_prefs - api_keys.xml	An API key or secret key was found in plaintext local storage. API keys grant access to external services and must not be embedded in client-side storage.
CRITICAL	Plaintext API Key	shared_prefs - api_keys.xml	An API key or secret key was found in plaintext local storage. API keys grant access to external services and must not be embedded in client-side storage.
CRITICAL	Plaintext API Key	shared_prefs - api_keys.xml	An API key or secret key was found in plaintext local storage. API keys grant access to external services and must not be embedded in client-side storage.

Figure 7: Rapport PDF généré côté client pour le scan 9998de76a2e5 (23/05/2026, 23:00:56 UTC). En-tête : Score 100/100, Niveau CRITICAL RISK, Critical 30 | High 49 | Medium 6 | Low 2. Table des findings : les premières entrées CRITIQUE montrent *Plaintext Auth Token* et *Plaintext API Key* localisés dans `shared_prefs - api_keys.xml`, avec description de l'impact et recommandation EncryptedSharedPreferences / Android Keystore.

6 Sécurité de la Plateforme

VaultDex traite des fichiers APK potentiellement malveillants soumis par plusieurs utilisateurs. Un modèle de sécurité en profondeur a été implémenté pour protéger la plateforme elle-même contre les attaques ciblant l'API et le pipeline d'analyse.

6.1 Authentification et Isolation Multi-Tenant

Tous les endpoints protégés requièrent un JWT valide signé HMAC-SHA256. Les tokens ont une expiration de 7 jours et contiennent l'UUID de l'utilisateur dans le claim `sub`. Les mots de passe sont hachés via `bcrypt` avec un facteur de coût adaptatif. Toutes les requêtes de lecture ou modification de scans filtrent systématiquement par `owner_id = :user_id`, implémenté au niveau de la couche CRUD dans `crud.py`, empêchant tout accès croisé entre utilisateurs.

6.2 Sécurité des Uploads

La validation des fichiers uploadés est triple : vérification des magic bytes APK (0x504B0304), contrôle de l'extension `.apk`, et limite de taille 100 Mo. Les fichiers sont renommés en `uuid4().apk` à la réception, empêchant le path traversal et les collisions de noms. La bibliothèque `slowapi` enforce un rate limit de 5 uploads par minute par adresse IP.

6.3 Protection contre les Injections

Injection SQL : toutes les requêtes passent par SQLAlchemy ORM avec requêtes paramétrées — aucune concaténation de chaînes SQL. **Attaques XXE** : le parsing des Shared Preferences XML utilise `defusedxml.ElementTree`, désactivant le traitement des entités externes par défaut, protégeant le scanner contre les APKs contenant des charges XXE. **En-têtes HTTP** : `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, Content Security Policy, et `Referrer-Policy: strict-origin-when-cross-origin` sont injectés sur toutes les réponses par le middleware FastAPI.

7 Déploiement

7.1 Prérequis et Configuration

La plateforme nécessite : Python 3.13+, Node.js 20+, Docker 24+, Genymotion Desktop 3.x et le SDK Android avec build-tools-34. Tous les chemins vers les outils externes sont configurés via variables d'environnement dans le fichier `.env`.

```

1  # Base de donnees
2  DATABASE_URL=postgresql+asyncpg://scanner:pass@127.0.0.1:5432/
   apk_scanner
3
4  # Broker Redis
5  REDIS_URL=redis://127.0.0.1:6379/0
6
7  # JWT
8  SECRET_KEY=votre-cle-secrete-256-bits
9  ACCESS_TOKEN_EXPIRE_DAYS=7
10
11 # Outils Android
12 ADB_PATH=/chemin/vers/platform-tools/adb
13 AAPT2_PATH=/chemin/vers/build-tools/34.0.0/aapt2
14 GENYMOTION_DEVICE_IP=192.168.56.101
15 GENYMOTION_SNAPSHOT_NAME=clean_state
16
17 # Repertoires de travail
18 UPLOADS_DIR=/chemin/vers/uploads
19 DUMPS_DIR=/chemin/vers/dumps
20
21 # CORS
22 ALLOWED_ORIGINS=http://localhost:5173

```

Listing 1: Variables d'environnement requises (`.env`).

7.2 Procédure de Démarrage Complète

```

1  # 1. Infrastructure (PostgreSQL + Redis)
2  docker-compose up -d
3  docker ps # Verifier : apk_scanner_db + apk_scanner_redis
4
5  # 2. Environnement Python
6  python -m venv .venv && source .venv/bin/activate
7  pip install -r requirements.txt
8  pip install "python-jose[cryptography]" "passlib[bcrypt]" "bcrypt
   <4.0.0"
9
10 # 3. Frontend
11 cd frontend && npm install && cd ..
12

```

```
13 # Terminal A : API FastAPI
14 uvicorn backend.api.app:app --host 127.0.0.1 --port 8000 --reload
15
16 # Terminal B : Worker Celery (concurrency=1 : emulateur unique)
17 celery -A backend.worker.celery_app worker --loglevel=info --
    concurrency=1
18
19 # Terminal C : Frontend
20 cd frontend && npm run dev
21
22 # Acces :
23 #   Dashboard      : http://localhost:5173
24 #   Swagger UI     : http://127.0.0.1:8000/docs
25 #   ReDoc           : http://127.0.0.1:8000/redoc
26 #   Health check   : http://127.0.0.1:8000/health
```

Listing 2: Procedure de demarrage complete de VaultDex.

8 Tests et Validation

8.1 Stratégie de Tests

Deux suites de tests couvrent les composants critiques : `test_engine.py` (tests unitaires du moteur d'analyse, sans émulateur requis) et `test_phase1.py` (tests d'intégration du pipeline Phase A, nécessitant Genymotion actif).

Table 9: Résultats de validation du moteur de règles

Artefact de test	Catégorie at- tendue	MASVS at- tendu	OK
Mot de passe en clair dans SharedPrefs settings.xml	CREDENTIALS	MASVS-STORAGE-1	✓
Token auth dans api_keys.xml	CREDENTIALS	MASVS-STORAGE-1	✓
Adresse email dans table SQLite users	PII	MASVS-STORAGE-1	✓
Hachage MD5 dans fichier cache .tmp	CRYPTOGRAPHY	MASVS-CRYPTO-1	✓
URL HTTP dans fichier config JSON	CONFIGURATION	MASVS-NETWORK-1	✓
allowBackup=true dans le manifeste	CONFIGURATION	MASVS-CODE-2	✓
debuggable=true dans le manifeste	CONFIGURATION	MASVS-CODE-2	✓

8.2 Métriques de Performance

Table 10: Métriques de performance mesurées sur VaultDex

Scénario	Valeur mesurée	Objectif
Latence GET /auth/me	< 15 ms	< 200 ms
Latence GET /scans/	< 30 ms	< 200 ms
Concurrence API (50 requêtes)	Sans dégradation	Réponse stable
Upload APK 16,28 Mo	< 3 s traitement API	< 10 s
Durée scan complète com.vaultdex.app	73 secondes	< 5 minutes
Génération rapport JSON	< 500 ms	< 2 s
Export PDF (87 findings)	< 1 s côté client	< 3 s

9 Contributions de l'Équipe

Table 11: Rôles et contributions de chaque membre de l'équipe

Membre		Rôle	Contributions
Abdelhakim Amhiraq		Développeur Backend	Factory FastAPI et lifespan, authentification JWT (<code>auth.py</code>), modèles ORM SQLAlchemy 2.0, opérations CRUD, middleware sécurité (CORS, rate limiting, headers HTTP), schémas Pydantic v2
Anas Aourik		Développeur Worker	Configuration Celery, implémentation pipeline deux phases (<code>scan_pipeline.py</code>), contrôleur ADB (<code>adb_controller.py</code>), gestion snapshots Genymotion, émission événements SSE
Lahsen Ait Oihmane		Développeur Moteur	Architecture moteur de détection à 28 règles, quatre modules de règles, cinq analyseurs spécialisés, algorithme de scoring, intégration defusedxml
Sultan Zeineddine		Développeur Frontend	Architecture SPA React 19, AuthContext JWT, interface SSE (<code>ScanProgressPage</code>), graphiques Recharts, export jsPDF, design system Tailwind-CSS v4, configuration Axios

10 Conclusion et Perspectives

10.1 Bilan

VaultDex représente une contribution concrète et innovante dans le domaine de l'automatisation de la sécurité mobile Android. En combinant analyse statique du manifeste APK (via `aapt2`) et analyse dynamique post-exécution réelle (via ADB sur émulateur Genymotion), le projet répond à un besoin identifié des équipes de développement mobile et des auditeurs de sécurité qui ne disposaient pas jusqu'alors d'une plateforme web unifiée, accessible et multi-tenant pour ce type d'audit.

L'architecture découplée — FastAPI pour l'API ASGI, Celery + Redis pour le traitement asynchrone, React 19 pour l'interface SPA, PostgreSQL 16 pour la persistance — assure la scalabilité, la maintenabilité et la robustesse de la solution face à des scans de durée variable. Le **pipeline en deux phases** avec fenêtre d'interaction manuelle constitue l'innovation architecturale centrale qui distingue VaultDex des approches d'automatisation classiques : tandis que l'analyse statique de `com.vaultdex.app` n'a identifié que 4 findings dans le manifeste, la phase dynamique a révélé 83 vulnérabilités supplémentaires dans les fichiers de stockage générés par l'usage réel de l'application.

Le **moteur de 28 règles** de sécurité, organisé en modules indépendants et extensibles (credentials, PII, cryptographie, configuration), a correctement classifié l'ensemble des cas de test de validation et détecté 87 findings dans l'APK de test en 73 secondes, bien en deçà de l'objectif de 5 minutes.

10.2 Perspectives d'Évolution

Plusieurs axes d'amélioration sont envisagés pour les versions futures de VaultDex :

1. **Analyse de trafic réseau** : Intégration d'un proxy MitM (mitmproxy) pour intercepter les communications HTTP/HTTPS de l'application durant la phase d'interaction, détectant les transmissions non chiffrées de données sensibles.
2. **Containerisation complète et déploiement cloud** : Remplacement de Genymotion Desktop par un émulateur Android headless (QEMU/AVD) conteneurisé pour un déploiement cloud sans dépendance logicielle sur le poste hôte.
3. **Intégration CI/CD** : Développement d'un plugin GitHub Actions / GitLab CI permettant d'intégrer VaultDex comme gate de sécurité dans les pipelines de déploiement continu des équipes de développement mobile.
4. **Rapport différentiel** : Comparaison automatique des findings entre deux scans successifs d'une même application pour mesurer l'évolution du score de sécurité au fil des versions.
5. **Support iOS** : Extension de l'architecture pour supporter l'analyse des archives IPA (iOS) via des simulateurs Xcode, en abstrayant la couche ADB derrière une interface commune de contrôle d'émulateur.

6. **Éditeur de règles personnalisées** : Interface d'autoring en navigateur permettant aux organisations de définir leurs propres patterns de détection métier sans modifier le backend Python.

References

- [1] OWASP Foundation (2024). *OWASP Mobile Security Testing Guide (MASTG)*, Version 2.0. <https://owasp.org/www-project-mobile-security-testing-guide/>
- [2] OWASP Foundation (2024). *OWASP Mobile Application Security Verification Standard (MASVS)*. <https://github.com/OWASP/owasp-masvs>
- [3] MobSF Team (2024). *Mobile Security Framework (MobSF)*. <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [4] S. Ramirez (2025). *FastAPI — Framework web moderne et rapide pour Python*. <https://fastapi.tiangolo.com>
- [5] Celery Project (2025). *Celery 5 — File de tâches distribuées*. <https://docs.celeryq.dev>
- [6] Android Developers (2025). *Android Debug Bridge (ADB)*. <https://developer.android.com/tools/adb>
- [7] Meta Open Source (2025). *React 19 — La bibliothèque pour interfaces utilisateur*. <https://react.dev>
- [8] PostgreSQL Global Development Group (2025). *Documentation PostgreSQL 16*. <https://www.postgresql.org/docs/16/>
- [9] SQLAlchemy (2025). *SQLAlchemy 2.0 — The Python SQL Toolkit and ORM*. <https://docs.sqlalchemy.org/en/20/>
- [10] Pydantic (2025). *Pydantic v2 — Validation de données avec les type hints Python*. <https://docs.pydantic.dev/latest/>
- [11] C. Heimes (2024). *defusedxml — Protection contre les exploits XML*. <https://pypi.org/project/defusedxml/>
- [12] Genymobile (2025). *Documentation Genymotion Desktop*. <https://docs.genymotion.com>

Article rédigé dans le cadre du projet académique VaultDex

Université Cadi Ayyad (UCA), Marrakech, 2025–2026

Encadrant : Professeur Lachgar — m.lachgar@uca.ac.ma